OPLOGS-07: Analytics & Dashboards

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 7 of 8 | **Created:** December 2025 | **Last Updated:** 10/12/2026

Aggregation, Time Series, and Visualization Queries

This notebook covers aggregation functions, time series analysis, statistical patterns, and dashboard-ready queries for log analytics.

Table of Contents

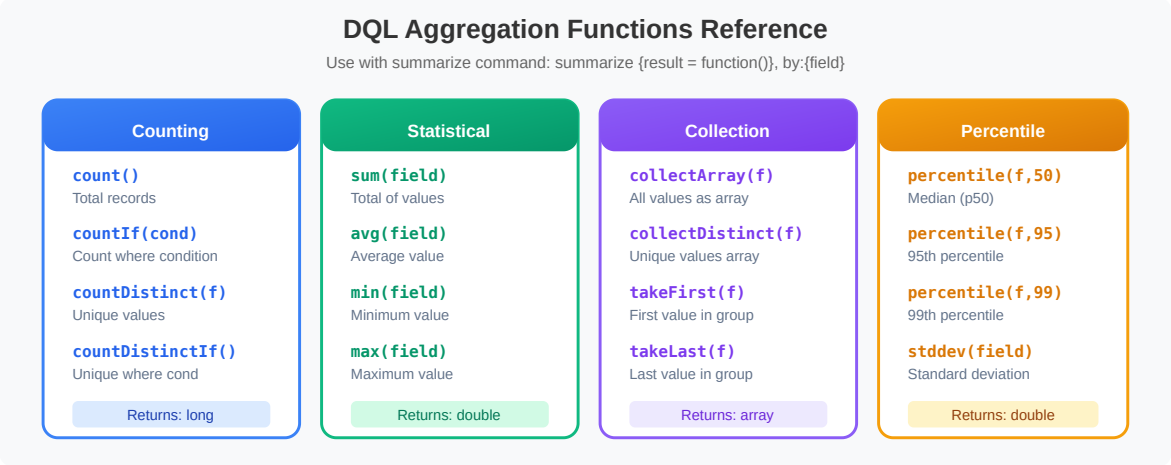
1. Aggregation Functions
2. Time Series Analysis
3. Statistical Analysis
4. Dashboard-Ready Queries
5. Trend Analysis
6. Log Pattern Analysis
7. Operational Dashboard Queries
8. 📝 Summary
9. ➡️ Next Steps
10. 📚 References

Prerequisites

- ✅ Access to a Dynatrace environment with log data

- ✅ Completed OPLOGS-01 through OPLOGS-06
- ✅ Familiarity with dashboard concepts

1. Aggregation Functions



Common Aggregation Functions

Function	Description	Example
`count()`	Count records	`summarize {total = count()}`
`countIf(condition)`	Conditional count	`countIf(loglevel == "ERROR")`
`countDistinct(field)`	Unique values	`countDistinct(k8s.pod.name)`
`sum(field)`	Sum values	`sum(bytes)`
`avg(field)`	Average	`avg(response_time)`
`min(field)`	Minimum	`min(timestamp)`
`max(field)`	Maximum	`max(timestamp)`
`takeFirst(field)`	First value	`takeFirst(content)`
`takeLast(field)`	Last value	`takeLast(content)`
`collectDistinct(field)`	Array of unique values	`collectDistinct(loglevel)`

🚨 Important: Named Aliases Required

```
// ✅ CORRECT - Named aliases
| summarize {count = count(), errors = countIf(...)}, by: {...}
| sort count desc

// ❌ WRONG - Anonymous aggregation can't be used in sort
| summarize count(), by: {...}
| sort count() desc  // ERROR!
```

```
// Basic aggregations
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    unique_hosts = countDistinct(dt.entity.host),
    unique_namespaces = countDistinct(k8s.namespace.name),
    unique_pods = countDistinct(k8s.pod.name)
  }
```

```
// Aggregations with grouping
// status == "ERROR" covers every error-class level (ERROR, SEVERE, CRITICAL,
FATAL, …);
// loglevel == "ERROR" alone misses SEVERE and the rest
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    errors = countIf(status == "ERROR"),
    warnings = countIf(status == "WARN"),
    info = countIf(status == "INFO")
  }, by: {k8s.namespace.name}
| sort total desc
| limit 15
```

```
// Error rate calculation
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    errors = countIf(status == "ERROR")
  }, by: {k8s.namespace.name}
| filter total &gt; 100  // Minimum sample size
| fieldsAdd error_rate_pct = round((errors * 100.0) / total, decimals: 2)
| sort error_rate_pct desc
| limit 15
```

2. Time Series Analysis

The `makeTimeseries` command creates time-bucketed data for trend visualization.

```
// Log volume over time (5-minute buckets)
fetch logs, from: now() - 6h
| makeTimeseries {log_count = count()}, interval: 5m
```

```
// Error trend over time
fetch logs, from: now() - 6h
| makeTimeseries {
    total = count(),
    errors = countIf(status == "ERROR"),
    warnings = countIf(status == "WARN")
  }, interval: 5m
```

```
// Time series by dimension (namespace)
fetch logs, from: now() - 6h
| filter isNotNull(k8s.namespace.name)
| makeTimeseries {
    log_count = count()
  }, by: {k8s.namespace.name}, interval: 10m
```

```
// Error rate time series by host
fetch logs, from: now() - 6h
| filter isNotNull(dt.entity.host)
| makeTimeseries {
    total = count(),
    errors = countIf(status == "ERROR")
  }, by: {dt.entity.host}, interval: 15m
```

3. Statistical Analysis

```
// Log volume statistics by source
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    first_seen = min(timestamp),
    last_seen = max(timestamp)
  }, by: {dt.openpipeline.source}
// duration ÷ duration yields a number; a long ÷ duration would return null
| fieldsAdd span = last_seen - first_seen
| fieldsAdd duration_hours = round(span / 1h, decimals: 1)
| fieldsAdd logs_per_hour = round(total / (span / 1h), decimals: 0)
| sort total desc
```

```
// Content-length statistics, including the 95th percentile
fetch logs, from: now() - 1h
| fieldsAdd content_length = stringLength(content)
| summarize {
    avg_length = avg(content_length),
    p95_length = percentile(content_length, 95),
    min_length = min(content_length),
    max_length = max(content_length),
    total_logs = count()
  }, by: {k8s.namespace.name}
| sort avg_length desc
| limit 15
```

```
// Hourly distribution analysis
fetch logs, from: now() - 24h
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {log_count = count()}, by: {hour_bucket}
| sort hour_bucket asc
```

```
// Daily distribution analysis
fetch logs, from: now() - 7d
| fieldsAdd day_bucket = bin(timestamp, 24h)
| summarize {
    log_count = count(),
    error_count = countIf(status == "ERROR")
  }, by: {day_bucket}
| sort day_bucket asc
```

4. Dashboard-Ready Queries

> 📊 **Reference queries moved to OPLOGS-99.** See **OPLOGS-99 § 9.1 Dashboard-Ready Queries** for single-value, time-series, breakdown, and percentile queries optimized for dashboard tiles. The teaching content for *how* dashboard queries are constructed lives in OPLOGS-05 (querying-parsing).
>
> 💰 **Cost-aware design — prefer a metric for queries that run on a schedule.** Dashboard tiles and alerts re-run their query on every render and evaluation. A `fetch logs … | summarize` behind a recurring tile pays the Grail **log Query** cost every time, whereas the same answer sourced from a metric does not — the Metrics powered by Grail **Query** dimension is always included at no additional charge, so metric reads never appear on the bill. For any recurring aggregate, check for an out-of-the-box metric first, and otherwise extract one from the log stream at ingest (OpenPipeline metric extraction — see OPLOGS-03 §3). Reserve `fetch logs` for one-shot investigation. **FAQ-09** covers the full decision and the DPS query economics.

5. Trend Analysis

```
// Compare current hour to previous hour
fetch logs, from: now() - 2h
| fieldsAdd period = if(timestamp &gt; now() - 1h, "current", else:
"previous")
| summarize {count = count()}, by: {period, loglevel}
| sort period asc, count desc
```

```
// New error patterns: first seen in the last 30 minutes of a 24-hour window
// (a 1-hour window would call any pattern "new" that simply had not occurred in that hour)
fetch logs, from: now() - 24h
| filter status == "ERROR"
// Collapse UUIDs, then digits: a raw prefix makes every timestamped line unique, and replacing digits alone
// leaves a UUID's hex letters, so every request ID becomes its own "new" pattern
// (validation tenant, 24h of errors: 30,852 digit-only signatures vs 223 with UUIDs collapsed, 10/12/2026)
| fieldsAdd error_sig = substring(replacePattern(replacePattern(content, "[0-9a-fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0, to: 80)
| summarize {
    count = count(),
    first_seen = min(timestamp)
  }, by: {error_sig, k8s.namespace.name}
| filter first_seen &gt; now() - 30m
| sort first_seen desc
| limit 15
```

```
// Volume anomaly: latest complete 15-minute bucket vs the 6-hour average, per namespace
// The window is aligned to bucket boundaries with bin(now(), 15m); otherwise the last
// bucket is still filling and every namespace looks like a drop.
fetch logs, from: bin(now(), 15m) - 6h, to: bin(now(), 15m)
| filter isNotNull(k8s.namespace.name)
| makeTimeseries {log_count = count()}, by: {k8s.namespace.name}, interval: 15m
| fieldsAdd baseline = arrayAvg(log_count), latest = arrayLast(log_count)
| filter baseline &gt; 0
| fieldsAdd ratio = round(toDouble(latest) / baseline, decimals: 2)
| fields k8s.namespace.name, baseline, latest, ratio
| sort ratio desc
| limit 20
```

6. Log Pattern Analysis

```
// Top log patterns (by content prefix)
fetch logs, from: now() - 1h
// UUIDs → "&lt;id&gt;", then digits → "#", so timestamps and IDs do not make every line its own pattern
| fieldsAdd pattern = substring(replacePattern(replacePattern(content, "[0-9a-fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0, to: 60)
| summarize {count = count()}, by: {pattern}
| sort count desc
| limit 25
```

```
// Unique log levels and statuses
fetch logs, from: now() - 1h
| summarize {
    loglevels = collectDistinct(loglevel),
    statuses = collectDistinct(status),
    sources = collectDistinct(dt.openpipeline.source)
  }
```

```
// Log diversity score (unique patterns per namespace)
fetch logs, from: now() - 1h
| fieldsAdd pattern = substring(replacePattern(replacePattern(content, "[0-9a-fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0, to: 50)
| summarize {
    total_logs = count(),
    unique_patterns = countDistinct(pattern)
  }, by: {k8s.namespace.name}
| fieldsAdd diversity_ratio = round((unique_patterns * 100.0) / total_logs, decimals: 2)
| sort diversity_ratio desc
| limit 15
```

7. Operational Dashboard Queries

> 🚦 **Reference queries moved to OPLOGS-99.** See **OPLOGS-99 § 9.2 Operational Dashboard Queries** for SLO-style queries: error budgets, top-N services by volume, per-host log distribution.

📝 Summary

In this notebook, you learned:

✅ **Aggregation functions** - count, countIf, countDistinct, sum, avg
✅ **Time series** - makeTimeseries with intervals
✅ **Statistical analysis** - hourly/daily patterns, percentiles
✅ **Dashboard queries** - Single values, charts, tables
✅ **Trend analysis** - Period comparison, anomaly detection
✅ **Pattern analysis** - Log clustering and diversity

➡️ Next Steps

Continue to **OPLOGS-08: Security & Data Protection** for data protection patterns.

📚 References

- Aggregation functions (DT docs)
- DQL aggregation commands (DT docs)
- Dashboards and Notebooks (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.